

## Lab Assignment

# Harmonic Motion

Due at the start of your next lab

## 1 Objectives

1. Implement a MATLAB function that uses selection statements.
2. Solve the equations for simple harmonic for the undamped, underdamped, critically-damped, and over-damped cases.
3. Graph multiple lines on the same plot.

## 2 Background

Harmonic motion is an important concept in all fields of engineering, including vibrational analysis. With *undamped* or *simple* harmonic motion, the system will oscillate forever at its resonant frequency once motion has been initiated. Although you have only seen undamped harmonic motion, real physical systems contain some *damping* that causes the amplitude of the oscillation to decay over time — otherwise we would have perpetual motion.

Some useful equations follow:

Undamped

Dynamical Force Equations:

$$\begin{aligned} F_{\text{net}} &= ma \\ -kx &= m\ddot{x} \\ \ddot{x} + \frac{k}{m}x &= 0 \end{aligned}$$

The solution of which is:

$$\begin{aligned} \omega_0 &= \sqrt{\frac{k}{m}} \\ A &= x_0 \\ B &= v_0/\omega_0 \\ x &= A \cos(\omega_0 t) + B \sin(\omega_0 t) \end{aligned}$$

Damped

Dynamical Force Equations:

$$\begin{aligned} F_{\text{net}} &= ma \\ -b\dot{x} - kx &= m\ddot{x} \\ \ddot{x} + \frac{b}{m}\dot{x} + \frac{k}{m}x &= 0 \end{aligned}$$

The solution of which is:

$$\begin{aligned} \omega_0 &= \sqrt{\frac{k}{m}} \\ \zeta &= \frac{b}{2\sqrt{mk}} \\ \omega_1 &= \omega_0 \sqrt{1 - \zeta^2} \end{aligned}$$

Case 1: Underdamped  $\zeta < 1$

$$\begin{aligned} A &= x_0 \\ B &= (\zeta\omega_0 x_0 + v_0)/\omega_1 \\ x &= e^{-\zeta\omega_0 t} (A \cos(\omega_1 t) + B \sin(\omega_1 t)) \end{aligned}$$

Case 2: Critically Damped  $\zeta = 1$

$$\begin{aligned} A &= x_0 \\ B &= \omega_0 x_0 + v_0 \\ x &= e^{-\omega_0 t} (A + Bt) \end{aligned}$$

Case 3: Overdamped  $\zeta > 1$

$$\begin{aligned} \gamma_1 &= \omega_0(-\zeta + \sqrt{\zeta^2 - 1}) \\ \gamma_2 &= \omega_0(-\zeta - \sqrt{\zeta^2 - 1}) \\ B &= -(\gamma_1 x_0 - v_0)/(\gamma_2 - \gamma_1) \\ A &= x_0 - B \\ x &= Ae^{\gamma_1 t} + Be^{\gamma_2 t} \end{aligned}$$

where

|                           |                                |                               |
|---------------------------|--------------------------------|-------------------------------|
| $x$ = displacement        | $m$ = mass                     | $k$ = spring constant         |
| $v$ = velocity            | $\omega_0$ = natural frequency | $\omega_1$ = damped frequency |
| $b$ = damping coefficient | $\zeta$ = damping ratio        |                               |

Note that the undamped case is the same as the underdamped case with  $\zeta = 0$ , as we should expect.

### 3 Requirements

1. Write a MATLAB function (in its own file) that implements all of the preceding cases:

```
function [x,t,damping] = harmonicMotion (m, k, b, x0, v0, tN, N)
% harmonicMotion
%   [x,t,damping] = harmonicMotion (m, k, b, x0, v0, tN, N)
%   computes the displacement x of a damped harmonic oscillator where:
% Inputs (all scalar):
%   m   = mass
%   k   = spring constant
%   b   = damping coefficient
%   x0  = initial displacement
%   v0  = initial velocity
%   tN  = final time (initial time = 0)
%   N   = number of points in time
% Outputs:
%   x   = vector of N displacement values
%   t   = vector of N time values
%   damping = a string containing 'undamped', 'underdamped', 'critical', or 'overdamped'
```

- (a) Check that all inputs are scalar values and that:
  - i.  $m$ ,  $k$ ,  $tN$ , and  $N$  are positive
  - ii.  $b$  is non-negative
  - iii.  $N$  is integer
- (b) Notice that we pass in the damping coefficient  $b$  (not  $\zeta$ ), so you will need to do the conversion in your function.
- (c) Since undamped is a special case of underdamped, it uses the same equations in your code ... but you will need to select the correct text label.
- (d) Do not use any **for** or **while** loops. Use vectors operations.
- (e) Only use the `.*` and `./` operators where it is *required*. Otherwise you *must* use the `*` and `/` operators. This makes your code easier to follow.
- (f) Note that you must return three things: the vector  $x$ , the vector  $t$ , and a string containing a word such as *underdamped*

2. Write a MATLAB script file (another file) that:

- (a) Generates four solutions for the four dampening cases using appropriate values for the damping coefficient while keeping the same mass and spring constant for each case so that you get a related set of curves.
- (b) So, although our function will solve *any* harmonic motion problem, we want to generate *specific* cases so that we get illustrative graphs. Any easy way to generate our four contrived cases is to specify a vector of  $\zeta$ 's in our script file and then convert these to  $b$ 's that we need to pass to the function. Note that, since `harmonicMotion` only handles scalar values, it will need to be called multiple times. You can do this using a `for` loop if you like.
- (c) Since the function `harmonicMotion` internally uses the variable names `m`, `k`, `b`, `x0`, `v0`, `tN`, and `N`, use *different names* in your script file. This isn't required by MATLAB, but we want to make the point that the variable names used inside the function are **not** known in the workspace.
- (d) Generate a first MATLAB figure that overlays four plots: 1) Undamped, 2) Underdamped, 3) Critically Damped, and 4) Overdamped. Your plot must have a legend that identifies the curves and your legend command must use the variable returned by the function, i.e., do not hard-code a name such as 'underdamped'.
- (e) Generate a second figure, plotting each of the four curves on separate subplots. Use *linkaxes* so that the plots use the same scales. See the example figures on the course web site.

## 4 Evaluation

1. Hand in your complete solution the problem (i.e., the function file and the script file) *with comments*. Save your MATLAB code into `.m` files and have comments that identify the problem being solved.
2. Show me your plots. You do not need to hand in the plots, just the code.